

Advertencias del IDE, carpetas target, cómo funcionan los tests y la línea REVISAR de ReportQueryTest

Documento explicativo · 7 secciones + 2 apéndices · generado el 7 de octubre de
2026

Este documento NO modificó nada del proyecto

Todo lo que sigue es resultado de **leer** el código y de ejecutar comandos de solo lectura (`validate`, `dependency:resolve`, `git ls-files`, `git check-ignore`). No se creó, no se borró y no se editó ningún archivo del repositorio. Cuando el documento propone un cambio, va marcado como **propuesta** para que decidas tú.

0. Las 5 preguntas y su respuesta en una línea

#	Tu pregunta	Respuesta corta	Se resuelve en
1	¿Por qué los YAML marcan Unknown property 'app' ?	La extensión de VS Code solo conoce las propiedades estándar de Spring Boot. Las tuyas son propias. No es un error: es falta de metadatos.	§1
2	¿Me falta una dependencia o una extensión?	Ninguna de las dos es obligatoria. Sí hace falta la extensión <code>Spring Boot Tools</code> para que exista la validación; la dependencia del procesador es opcional y solo mejora el autocompletado.	§1.2 y §1.6
3	¿Qué es <code>target-test-classes</code> ?	Es la carpeta de salida de la clase de tests del Spring Tool Suite (Eclipse). En ese proyecto se compiló fuera de Maven. <code>target/test-classes</code> es la carpeta correcta.	§3
4	¿ÍVa dentro de <code>test</code> o al <code>ignore</code> ?	Al ignore . No va dentro de <code>src/test</code> : ahí solo va código fuente. Y hoy no está ignorada: hay 1 archivo ya versionado en git.	§3.5 y §4
5	¿Cómo funcionan los tests y cómo escribo míos?	4 familias distintas: Mockito puro, <code>@DataJpaTest</code> , <code>@WebMvcTest</code> e integración. Cada una a una capa distinta.	§5

Y una 6 que no preguntaste pero salió de la revisión

La advertencia del `pom.xml` y la línea `//REVISAR` de `ReportQueryTest` están documentadas en §2 y §6. Y apareció un hallazgo que no está en ninguna de tus preguntas: **hay un archivo `.class` compilado dentro de git** (§3.5).

1. El aviso Unknown property 'app'

1.1 El aviso exacto

Lo que ves marcado en amarillo es el código `vscode-spring-boot (YAML_UNKNOWN_PROPERTY)`. Traducido por la extensión:

Qué significa literalmente

«Hay una propiedad YAML llamada `app` que yo no conozco.»

La extensión no está diciendo que esté mal escrita. Está diciendo **para mí ese nombre no existe**. Y tú lo dice respetuosamente en amarillo, no en rojo.

1.2 Por qué aparece (el mecanismo exacto)

Spring Boot lleva dentro un archivo llamado `spring-configuration-metadata.json`. Es su «lista de propiedades conocidas». El autocompletado, la documentación al pasar el mouse y esta validación salen de ahí.

Ese JSON lo genera el propio Spring Boot y **solo contiene lo que Spring Boot entiende**:

```
spring.datasource.url      <- el framework lo define
server.port                <- el framework lo define
spring.jpa.hibernate.ddl-auto <- el framework lo define
app.frontend-url           <- TUYA: el framework no tiene ni idea
```

De ahí la regla que resuelve el 90% de las dudas: **propiedad desconocida no significa propiedad mal escrita**. Significa propiedad *propia*.

Y por qué te pasó ahora y antes no: en esta última sesión (la del correo, 2026-10-07) se agregó el bloque `app.mail.*` en el plan de Resend y se tocó el YAML. Al haber más claves propias bajo `app:`, hay más líneas donde marcar. El aviso es el mismo de siempre; solo cambió la cantidad de filas.

1.3 Inventario real: qué propiedades tienes y quién las lee

Esto lo verifiqué buscando en todo el código quién lee cada propiedad. El bloque `app:` de `application.yaml` tiene **4 claves**:

Propiedad (YAML línea)	¿Alguien la lee?	Dónde y cómo
<code>app.frontend-url</code> (línea 93)	Sí	<code>AuthImpl.java:39</code> con <code>@Value("\${app.frontend-url}")</code> . Arma el enlace del correo de recuperación.
<code>app.seed-bootstraps</code> (línea 99)	Sí	Los 4 bootstraps: <code>AdminBootstrap:42</code> , <code>PermissionBootstrap:27</code> , <code>RoleBootstrap:28</code> , <code>RolePermissionBootstrap:51</code> .
<code>app.upload-dir</code> (línea 106)	Sí	<code>WebConfig:27</code> y <code>FileStorageService:46</code> . Dónde caen las imágenes de productos.
<code>app.upload-url</code> (línea 110)	Sí	<code>ProductMapper:22</code> . URL base que el frontend usa para mostrar las imágenes.

Dato tranquilizador

Las 4 propiedades del bloque `app:` están en uso. No hay ninguna llave muerta. El aviso es puramente cosmético: el YAML es correcto y la aplicación funciona.

El otro bloque propio es `payment.terminal.*` (líneas 116-131). Ese sí está declarado como clase tipada, y es el único caso del proyecto que *sí* se puede describir solo:

```
// config/TerminalConfig.java
@Component // linea 25
@ConfigurationProperties(prefix = "payment.terminal") // linea 26
@Data // Lombok: getters Y setters
public class TerminalConfig {
    private String type; // <- payment.terminal.type
    private int port; // <- payment.terminal.port
    ...
}
```

1.4 La diferencia clave: @Value vs @ConfigurationProperties

Esta es la parte que decide qué solución te sirve, así que te la saltes. Hay **dos maneras** distintas de leer configuración personalizada en Spring Boot:

	@Value (inyección de valor)	@ConfigurationProperties (binding a clase)
Cómo se escribe	@Value("\${app.frontend-url}") pegado a un campo	Anotación en la clase + una propiedad por campo con getter y setter
Qué sabe el framework	Nada. Solo copia el texto. Puede ni comprobar que la propiedad exista.	Todo. Nombres, tipos, valores por defecto, validación, conversión de texto a número o booleano.
¿Genera metadatos?	NO — el procesador de Spring ignora por completo el @Value.	Sí — de aquí salen los metadatos.
Ejemplo en tu proyecto	app.frontend-url, app.seed-bootstraps, app.upload-dir, app.upload-url	payment.terminal.* (TerminalConfig)

Consecuencia directa y no obvio

Si agregas la dependencia `spring-boot-configuration-processor` esperando que desaparezcan los avisos amarillos, **só funcionará para `app.*`**: esas 4 propiedades se leen con @Value, y el procesador no las ve. Lo único que arreglaría con esa dependencia es el bloque `payment.terminal.*`.

Tu pregunta era «¿me falta una dependencia?». La respuesta correcta es: **no te falta ninguna**. Esa dependencia es la solución equivocada para este caso.

1.5 Solución 1 — silenciar el aviso (recomendada)

Es la solución que corresponde al problema real: no hay error, solo molestia visual. Se hace en los ajustes del espacio de trabajo, **sin tocar el repositorio**.

Paso 1. Crear la carpeta `.vscode` en la raíz del proyecto (hoy **no existe**; además `.vscode/` está en el `.gitignore`, así que el archivo no se versiona y es solo tuyo).

Paso 2. Crear dentro `.vscode/settings.json` con:

```
{
  "spring-boot.ls.problem.application-yaml.YAML_UNKNOWN_PROPERTY": "IGNORE"
}
```

Paso 3. Recargar la ventana (Ctrl+Shift+P → Developer: Reload Window). Los amarillos desaparecen.

Si prefieres no perder la señal y solo bajarla de intensidad, el mismo ajuste acepta estos valores:

Valor	Qué hace	¿Cuándo?
'IGNORE'	Ni se muestra el aviso.	El proyecto compila, los tests pasan y las propiedades se leen. Es el estado real de las cosas.
'warning'	Se muestra como advertencia normal (sin el color especial de metadatos faltantes).	Si quieres enterarte de que apareció una propiedad <i>de verdad</i> desconocida.
'error'	Se muestra como error.	Nunca aquí: te llenaría la pantalla de errores falsos.

1.6 Solución 2 — la dependencia del procesador (opcional)

Solo tiene sentido si lo que quieres es tener **autocompletado y descripciones** al escribir, no solo silence el aviso. Y recordemos: solo aplica a @ConfigurationProperties.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
```

Qué hace técnicamente: al compilar, Maven corre un procesador de anotaciones que lee las clases con `@ConfigurationProperties` y escribe este archivo:

```
target/classes/META-INF/spring-configuration-metadata.json

{
  "groups": [
    { "name": "payment.terminal",
      "type": "com.erikjarquin.compras.config.TerminalConfig" }
  ],
  "properties": [
    { "name": "payment.terminal.type",
      "type": "java.lang.String",
      "description": "Modo de operacion: PHYSICAL o SIMULATED" },
    ...
  ]
}
```

La extensión de VS Code lee ese archivo del target y por eso se acaban los avisos. Con tu proyecto tal como está, el resultado sería: se documentaría `payment.terminal.type` (porque `TerminalConfig` tiene getters y setters gracias a `@Data`), y los avisos de `app.*` seguirían ahí.

Detalles que conviene saber antes de decidir:

- bullet **Es invisible en tiempo de ejecución:** es un `<optional>true</optional>` de compilación, no entra al `.jar` que se despliega. No cambia el comportamiento de la app.

- bullet **Solo hay una clase** `@ConfigurationProperties` en todo el proyecto (`TerminalConfig`). El resultado sería pequeño: unas 10 propiedades de la terminal.

- bullet **El JSON aparece en target/**, que está en el `.gitignore`. Cada compilación limpia lo regenera.

- bullet **Si algún día escribes mal el nombre de un campo en TerminalConfig**, la app **falla al arrancar** con un mensaje claro. Ese es el beneficio real: convierte un error silencioso en un error ruidoso.

1.7 Solución 3 — declarar el prefijo app como clase

Si en algún momento te molesta de verdad la validación y quieres autocompletado para **todas** las propiedades propias, la única forma es dejar de usar `@Value` para ellas y declararlas en una clase. Es la única solución de las tres que cubre `app.*`.

Ojo: la solución 3 es la más invasiva de las tres

Advertencia importante: hoy un error de tecleo en el YAML pasa desapercibido hasta que `AuthImpl` manda un correo con un enlace roto. Si lo conviertes en clase tipada, la app **no arranca** y el error aparece en el log. Por eso el orden de preferencia es: 1) silenciar el aviso, 2) si molesta mucho, cambiar a clase tipada; la dependencia del procesador sola **no** es una solución completa.

1.8 Lo que NO debes hacer

No hagas esto	Por qué
Borrar o comentar el bloque app: del YAML	Las 4 propiedades están en uso. Si las borras, la app no arranca : AuthImpl y los 4 bootstraps piden app . * y no encuentran nada. El aviso desaparece y el proyecto deja de funcionar.
Mover app: dentro de spring:	Escribirías cosas que no son de Spring bajo el prefijo de Spring. Funciona, pero si algún día actualizas Spring Boot, el framework podría interpretar esas claves y complicar el diagnóstico.
Renombrar las propiedades para que coincidan con el 'estándar'	app.mail.provider no existe como propiedad estándar. No hay forma de «acomodarlo» sin renombrarlo tú, y renombrarlo toca código.
Dejar el aviso sin resolver si molesta	Es la opción por defecto y es: no rompe nada. Solo conviene silenciarlo si te distrae mientras lees el YAML.

1.9 Cómo confirmar que quedó bien

Si aún dudas de si el aviso es solo cosmético, haz esta prueba manual: arranca la aplicación y revisa que las propiedades se estén leyendo de verdad.

```
# 1. Los tests siguen en verde (esto no lo Comprueba el YAML, comprueba Java)
.\mvnw.cmd test -Dtest='!ComprasApplicationTests'

# 2. La app arranca -> los @Value se resolvieron sin excepcion
.\mvnw.cmd spring-boot:run

# 3. Prueba de fuego del bloque app:
#   pide recuperacion de contraseña: si app.frontend-url estuviera mal,
#   el correo saldría con un enlace roto (o la app ni siquiera levantaría).
```

Veredicto

Si la app levanta y los 294 tests pasan, el YAML está bien. Punto. El amarillo del editor es información, no un defecto.

2. La advertencia del pom.xml

2.1 Maven no reporta ninguna advertencia

Antes de diagnosticar nada, corrí Maven sobre el proyecto. El resultado real:

```
> .\mvnw.cmd validate
[INFO] Scanning for projects...
[INFO] -----< com.erikjarquin:compras >-----
[INFO] Building compras-backend 0.0.1-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----

> .\mvnw.cmd -o dependency:resolve      (buscando WARNING / "version for" / "missing")
(sin salida: cero advertencias)
```

Esto cambia el diagnóstico por completo

Maven, que es quien de verdad construye el proyecto, **no se queja de nada**: ni al leer el modelo, ni al resolver las 25 dependencias. Por lo tanto la advertencia amarilla **no viene de Maven**: la pone el editor (la extensión de Java o el validador XML de VS Code) aplicando reglas que Maven no aplica. Traducción: **es ruido del editor, no un problema del pom.xml**.

2.2 Los puntos del pom que suele marcar el editor

Con la información de arriba, las únicas zonas del pom.xml que un editor puede señalar son estos bloques, todos generados automáticamente al crear el proyecto:

Línea	Código	Por qué se marca
16	<url/>	Elemento vacío. Maven lo creó así y no molesta, pero un validador de esquema lo señala como valor faltante.
17-19	<licenses><license/></licenses>	Sin <name> ni <url>. Solo importa si publicas el artefacto en un repositorio Maven público, que aquí no es el caso.
20-21	<developers><developer/></developers>	Idem: datos del autor vacíos. Sin efecto en la compilación.
22-28	bloque <scm>	Repositorio de código vacío. Igual que el anterior.

2.3 «optional» no es una advertencia

Si la señal que ves señala el <optional>true</optional> de Lombok (línea 36), conviene saber qué significa, porque es frecuentemente malinterpretado:

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

Lo que la gente cree	Lo que significa de verdad
«Es opcional, quizálo»	No. Se usa en tiempo de compilación para generar los getX()/setX() de @Data. TerminalConfig depende de eso.

Lo que la gente cree	Lo que significa de verdad
«No se agrega al jar final, así que da igual»	Casi. No aparece en el <code>.jar</code> de la aplicación, pero sí se propaga a quien dependa de ti como librería. Para una app que se ejecuta sola, <code><optional></code> es lo correcto.
«Está avisando que falta algo»	No. Es una declaración de intención, no un error. Maven no dice nada (verificado).

2.4 Cómo leer el aviso exacto (para no adivinar)

Un editor puede mostrar el mismo `pom.xml` con advertencias distintas según la máquina y según la extensión instalada. Para saber cuál es el tuyo:

- bullet **Panel Problems** (Ctrl+Shift+M): aparece el mensaje completo, no solo el triángulo.

- bullet Ctrl+Shift+P → Problems: Show All Problems, o hacer clic sobre el triángulo amarillo para ver el texto en el editor.

- bullet **Leer la regla:** las advertencias de Maven con `validate` salen precedidas de `[WARNING]`. Como no hay ninguna, lo que ves lo pone el editor.

- bullet **Probar sin el editor:** si `.\mvnw.cmd clean test -Dtest='!ComprasApplicationTests'` termina en `BUILD SUCCESS` con los 294 tests en verde, el `pom.xml` está correcto.

Recomendación

No toques el `pom.xml`. Si el aviso te molesta y es de los bloques vacíos, la opción trivial es borrarlos (líneas 16-28), que no cambian el comportamiento. Pero como Maven no reporta nada, la alternativa más honesta es dejarlo: un `pom` con bloques de metadatos vacíos es inofensivo y es lo que genera el propio Spring Initializr.

3. Para qué sirve cada carpeta

3.1 El mapa completo

Este es el único árbol que importa. Todo lo demás son derivados:

```
Compras_Backend/          <- raiz del repositorio
|
|-- pom.xml                <- lo LEES tu. Version, dependencias, plugins.
|-- AGENTS.md              <- contexto del proyecto (este doc lo complementa)
|-- mvnw.cmd / mvnw        <- Maven empaquetado (no necesitas Maven instalado)
|
|-- src/main/java/...      <- CODIGO DE LA APLICACION
|   controller/ service/ service/impl/ repository/ model/ mapper/ config/ ...
|
|-- src/main/resources/
|   application.yaml        <- configuracion (SI se versiona)
|   application-local.yaml  <- tus secretos (NO se versiona, esta en .gitignore)
|   db/migration/V1__init.sql ... <- migraciones Flyway
|
|-- src/test/java/...      <- CODIGO DE LOS TESTS (aquí va tu test)
|   repository/ controller/ service/ util/ config/
|   ReportQueryTest.java    <- el archivo de la seccion 6
|
|-- target/                <- GENERADO. Todo lo de aquí se puede borrar.
|   classes/               <- tu app compilada (.class)
|   test-classes/          <- TUS TESTS compilados (.class)    <- ESTE es el bueno
|   generated-sources/     maven-status/ surefire-reports/ ...
|
|-- target_test-classes/   <- GENERADO, pero HUERFANO y MAL IGNORADO (<= seccion 3.4)
|   com/erikjarquin/compras/... <- 49 .class, incluido ReportQueryTest.class
|
|-- docs/                  <- PDFs como este
|-- uploads/               <- imagenes subidas (ignorado)
|-- .gitignore
```

La única regla que resume todo el capítulo

src/ es lo que escribes tú y se versiona. target/ es lo que se compila y se borrar. Un `.class` es un archivo binario generado a partir de tu `.java`. Si está en git, es como subir el PDF de un libro que ya tienes en el código fuente: pesa, no se revisa, y genera conflictos de diff ilegibles.

3.2 target/

Es el directorio de trabajo de Maven. Lo crea al compilar y lo puede borrar entero sin pérdida alguna, porque se regenera:

Subcarpeta	Qué contiene	¿Se puede borrar?
classes/	Tu aplicación compilada: los <code>.class</code> de cada <code>@Service</code> , <code>controller</code> , <code>entity</code> . Además copia <code>application.yaml</code> y los <code>.sql</code> de Flyway aquí dentro.	Sí. Es lo que borra <code>clean</code> .
test-classes/	Lo mismo pero de los tests: los <code>.class</code> de los 28 archivos de <code>src/test/java</code> . Ahí aparece el <code>.class</code> que Surte de <code>ReportQueryTest.java</code> .	Sí.
generated-sources/	Código que genera el framework (implementaciones de interfaces de JPA, etc.).	Sí. Se regenera.
surefire-reports/	Los resultados de la última corrida de tests: un <code>.txt</code> y un <code>TEST-*.xml</code> por clase. Cuando un test falla, aquí está el detalle.	Sí, pero entonces pierdes el diagnóstico.

Subcarpeta	Qué contiene	¿Se puede borrar?
maven-status/	Bookkeeping interno de Maven (qué recompiló).	Sí.

3.3 target/test-classes/ — la correcta

Esta es la carpeta que Maven usa para los tests, y la única relacionada con tu pregunta. Cuando compilas, ocurre esto:

```
src/test/java/com/erikjarquin/compras/repository/ReportQueryTest.java
|
|   javac (dentro de maven-compiler-plugin:testCompile)
v
target/test-classes/com/erikjarquin/compras/repository/ReportQueryTest.class
|
|   el plugin surefire lo carga con ClassLoader y ejecuta los @Test
v
target/surefire-reports/com.erikjarquin.compras.repository.ReportQueryTest.txt
```

Verifiqué que en tu máquina actual la carpeta existe pero está **vacía** (0 archivos). Eso es normal: `mvn validate` no compila nada. Se llena cuando compilas o corres tests. Si borras la carpeta y corres `mvn test`, Maven la recrea: por eso `target/` no vale nada.

3.4 target_test-classes/ — el huérfano

Esta carpeta **no es de Maven**. Está en la raíz del proyecto, al nivel de `target/`, y contiene 49 archivos `.class` (0.33 MB).

¿De dónde sale el nombre?

El nombre con **guion bajo** es la pista. Spring Tool Suite 4 (el IDE de Spring basado en Eclipse) usa por defecto dos carpetas de salida distintas:

```
Spring Tool Suite 4 (Eclipse)
  entrada de la app  -> target/classes/           (con guion bajo)
  entrada de los test -> target_test-classes/      (con GUIÓN BAJO)
                        ^
                        esto no es Maven; Maven jamás crea un directorio así
```

Evidencia: en **tu proyecto** no hay ninguna configuración que pida esa carpeta. Lo comprobé:

- El `pom.xml` **no** declara `<testOutputDirectory>` (lo leí completo: 179 líneas, sin eso).

- `vscode/` **no existe**, así que no hay ajustes del editor que lo indiquen.

- No hay archivos de Eclipse (`.project`, `.classpath`, `.settings/`), aunque `.gitignore` los contempla (herencia de cuando el proyecto sí se jugaba con STS).

- Maven solo creó `target/` y sus subdirectorios, nunca uno con guion bajo a ese nivel.

Conclusión: algún compilación hecha desde un editor heredado de Eclipse escribió ahí. Es un residuo, y su contenido es exactamente el mismo que `target/test-classes/`: los tests compilados por otro camino.

Lo que hay adentro

```
target_test-classes/com/erikjarquin/compras/
  ComprasApplicationTests.class
  config/          controller/      repository/
  service/         util/
-> 49 archivos .class en total, 0.33 MB
```

Incluye esto, que es el que te interesa por la línea "REVISAR":
target_test-classes/.../repository/ReportQueryTest.class

3.5 El hallazgo serio: está dentro de git

Aquí está lo que hay que mirar. Un .class ya está **versionado**. Las pruebas que corrí:

```
> git check-ignore -v target_test-classes
exit=1          <- 1 = NO hay regla que lo ignore

> git ls-files target_test-classes | Measure-Object
1              <- hay 1 archivo RASTREADO (ya en el repo)

target_test-classes/com/erikjarquin/compras/repository/ReportQueryTest.class

> git log --diff-filter=A -- target_test-classes
252008e Solo falta rediseñar modulo de correo

> git status --short
 M target_test-classes/com/erikjarquin/compras/repository/ReportQueryTest.class
?? target_test-classes/com/erikjarquin/compras/ComprasApplicationTests.class
?? target_test-classes/com/erikjarquin/compras/config/
?? target_test-classes/com/erikjarquin/compras/controller/
?? target_test-classes/com/erikjarquin/compras/repository/ProductRepositoryTest.class
?? target_test-classes/com/erikjarquin/compras/repository/ProviderRepositoryTest.class
?? target_test-classes/com/erikjarquin/compras/repository/UserRepositoryTest.class
?? target_test-classes/com/erikjarquin/compras/service/
?? target_test-classes/com/erikjarquin/compras/util/
```

Qué significa cada línea:

Símbolo	Significado	Qué hacer
M (modificado)	Está en el último commit y además cambió desde él. O sea: recompilaste ReportQueryTest.java y el binario quedó con diferencias.	Sacar del índice de git.
?? (sin seguimiento)	Archivos nuevos que git todavía no conoce. Si haces git add . se van al commit.	Ignorar la carpeta antes de que eso ocurra.

Cómo pasó exactamente

En el commit 252008e («Solo falta rediseñar modulo de correo», la sesión del 2026-10-07) se añadió el comentario `//REVISAR ESTA LÍNEA a ReportQueryTest.java`, se compiló desde un editor que escribía en `target_test-classes/`, y el .class compilado de ese test se fue al commit. **El binario del archivo que querías revisar ya está en tu repositorio**. Esa es la razón de que ahora te aparezca la carpeta con tanto interés: no es nueva, es que la están viendo.

3.6 src/test/java/ — dónde va un test

Esta carpeta es la única donde debe vivir código de test. Ahí hay 28 archivos Java, organizados por la capa que prueban, replicando los paquetes de `src/main`:

```
src/test/java/com/erikjarquin/compras/
|
|-- repository/      ReportQueryTest, UserRepositoryTest, ProductRepositoryTest,
|                    ProviderRepositoryTest      <- capa de datos (H2)
|-- controller/      ProductControllerTest, SaleControllerTest, ... (12)
|                    <- capa web (MockMvc)
|-- service/          ProductImplTest, PurchaseImplTest, SaleImplLifecycleTest, ...
|                    <- logica de negocio (Mockito)
|-- config/           RolePermissionBootstrapTest
|-- util/             InputValidatorTest
`-- ...              (mismo package que en src/main: si AuthImpl esta en
                    service/impl, su test tambien)
```

Regla	Por qué
El package del test es idéntico al de la clase probada	Si AuthImpl está en service/impl, su test va en src/test/java/.../service/impl/. Los tests privados solo se ven así.
El nombre acaba en Test	Es lo que usa Surefire para detectarlos. MiClaseTests (con s) también funciona, pero la convención del proyecto es ClaseTest.
Nún .class aquí dentro	Si ves un .class en src/, el compilador está escribiendo en el lugar equivocado. En src/test/ solo hay .java.

3.7 El .gitignore y por qué no atrapó la carpeta

Tu .gitignore empieza así (líneas 1-4):

```
target/
.mvn/wrapper/maven-wrapper.jar
!**/src/main/**/target/
!**/src/test/**/target/
```

Por qué target/ no protege a target_test-classes/

Es la parte que más cuesta ver. Una regla de .gitignore que termina en / es un **nombre de carpeta exacto**: coincide con una carpeta llamada target y ya está.

Ruta	¿Coincide con la regla target/?	¿Se versiona?
target/	Sí (mismo nombre exacto)	No
target/test-classes/	Sí (está dentro de una ignorada)	No
target_test-classes/	NO (se llama target_test-classes, con guion bajo: es otro nombre)	Sí → ya está en git
target2/	No	Sí
src/test/target/	Sí, pero la línea 4 lo des-ignora	Sí

Y las dos negaciones de las líneas 3-4

Con **!**/src/main/**/target/** y **!**/src/test/**/target/**, si algún día apareciera una carpeta target dentro de src/, **git la versionaría** (el ! es negación: «este caso sí **sí** se ignora»).

En tu proyecto hoy no hay ninguna carpeta target dentro de src/, así que esas dos líneas no están causando daño algún. Se pueden dejar; no son el problema. El problema es que **falta la regla para target_test-classes/**.

3.8 Plan de limpieza (propuesta; nada ejecutado)

Por si decides actuar. Son pasos separados, en este orden, y cada uno verificable:

#	Paso	Comando	Por qué en este orden
1	Añadir la regla de ignorado	Editar <code>.gitignore</code> y añadir la línea <code>target_test-classes/</code> (o <code>target*/</code> para cubrir cualquier variante)	Primero se cierra la puerta. Si empiezas borrando archivos, el <code>git add</code> . del siguiente paso reintroduce todo.
2	Confirmar que ya no se rastrea	<code>git check-ignore -v target_test-classes/ComprasApplicationTests.clas</code> <code>s</code>	Debe imprimir la regla y salir con código 0. Si sigue saliendo 1, la regla no se aplicó.
3	Sacar lo ya versionado sin borrarlo	<code>git rm -r --cached target_test-classes</code>	Deja de estar en el índice pero el archivo sigue en tu disco. Es el primer paso de cualquier limpieza: no pierdes nada.
4	Borrar la carpeta del disco	Eliminar la carpeta <code>target_test-classes/</code> entera	Ya es innecesaria. Los tests se siguen compilando en <code>target/test-classes/</code> .
5	Evitar que vuelva	Compilar desde la terminal con <code>.\mvnw.cmd clean test</code> en lugar de desde el editor	Si vuelve a aparecer, el editor sigue configurado para escribir ahí. Raíz: <code><testOutputDirectory></code> dentro de esa configuración.
6	(Opcional) Limpiar el historial	<code>git filter-repo</code>	Saca el <code>.class</code> del commit 252008e. Es el mismo paso que ya necesita el proyecto para purgar secretos (AGENTS.md lo advierte). Se puede hacer junto.

Lo que el paso 3 NO hace

`git rm --cached` **no borra el archivo de tu disco**: solo lo saca del índice para el próximo commit. Por eso el paso 3 va antes que el 4, y por eso conviene hacer un `git status` después de cada uno. Si ejecutas `git rm -r sin --cached`, borras el archivo y el cambio queda registrado como una eliminación.

4. Respuesta directa: ¿dentro de test o en el ignore?

Ninguna de las dos: **las dos opciones que se plantean son la misma y están bien**. No es «dentro de test o ignore» como si fueran alternativas excluyentes.

Pregunta	Respuesta
¿Va dentro de <code>src/test/</code> ?	No . En <code>src/test/</code> solo va código fuente: archivos <code>.java</code> que escribes tú. Un <code>.class</code> es un archivo compilado; meterlo allí mezcla dos naturalezas distintas en la misma carpeta y <code>git status</code> mostrará 49 archivos modificados cada vez que compiles.
¿Va en el <code>.gitignore</code> ?	Sí , pero además hay que sacarlo del índice de git, porque hoy un archivo ya está versionado: ignorar no des-versiona. Y una regla solo no basta si <code>.gitignore</code> se committed después; para eso está la opción más simple de borrar la carpeta del disco.
¿Y dónde compila entonces?	En <code>target/test-classes/</code> , que ya está cubierto por la regla <code>target/</code> de la línea 2 de tu <code>.gitignore</code> . Ahí es donde Maven deja los tests compilados cuando corres <code>mvn test</code> .

Resumen en una frase

El código del test va en `src/test/java/`; el resultado de compilarlo va en `target/test-classes/`, que ya está ignorado; y `target_test-classes/` no debería existir — si el `.gitignore` la reconociera como lo que es (una carpeta generada), el aviso desaparecería solo.

5. Cómo funcionan los tests (guía para aprender)

5.1 Qué es un test y qué no es

Un test es una pregunta que le haces al código y que se contesta sí/no. Nada más. No es una demostración ni un documento.

Por qué existen en un negocio como el tuyo es concreto: tu `AGENTS.md` registra al menos cuatro bugs que costaron tiempo y datos. El más caro:

```
Compra 10 -> se venden 5 -> se cancela la compra
ProductImpl.cancel() hacia: stock = max(5 - 10, 0) = 0
El cliente recibio un 200 OK.

Con 5 unidades ya vendidas, el stock real era 0. Correcto por casualidad.
PERO: si se hubieran vendido 3, el stock real era 2 y el sistema guardaba 0.
Inventario FALSO, sin error, sin logs, sin que nadie lo notara.

El test que blinda eso (PurchaseImplTest):
    verify(productRepository, never()).save(any());
-> "no basta con lanzar la excepcion: hay que verificar que NADA se escribio"
```

Ese test es la razón de ser de toda la práctica. Un bug que escribe datos falsos y devuelve «todo bien» no se encuentra probando a mano: se encuentra con una pregunta automática que alguien escribió una vez y que vuelve a preguntar en cada build.

5.2 Anatomía de un test: las 3 partes

Todo test, sin excepción, tiene la misma estructura. Tomalá del código real de tu proyecto (`ReportQueryTest.java:133`):

```
@Test
void sumSalesByDay_agrupaPorDiaSoloAprobadas() {          // 3. el nombre: que se prueba
    // ----- GIVEN (preparar)
    // (el @BeforeEach ya guardo las ventas; aqui no hace falta mas)

    // ----- WHEN (actuar)
    List<Object[]> rows = saleRepository.sumSalesByDay(
        from, to, PaymentStatus.APPROVED);

    // ----- THEN (afirmar)
    assertThat(rows).hasSize(2);
    assertThat(rows.get(0)[1]).isEqualTo(new BigDecimal("10.00"));
    assertThat(rows.get(1)[1]).isEqualTo(new BigDecimal("15.00"));
}
```

Parte	Qué es	Cómo se reconoce en el código
Given	Dejar el mundo en un estado conocido	Los <code>@BeforeEach</code> , los mocks, los <code>save()</code> del seed.
When	Ejecutar lo que se quiere probar	La llamada al método. Una sola línea (o una muy corta). Si hay varias llamadas, casi siempre son dos tests.
Then	Comprobar el resultado	Los <code>assertThat(...)</code> . Deben poder fallar: si un <code>assert</code> no puede fallar, el test no prueba nada.

El nombre del test es la documentación. `sumSalesByDay_agrupaPorDiaSoloAprobadas` se lee como una afirmación: «sumSalesByDay agrupa por día y solo toma las aprobadas». Cuando falle en 6 meses, esa línea dice qué se rompió sin que haya que abrir el archivo. Por eso el proyecto escribe los nombres así (en español, con `_` separando el método del comportamiento).

5.3 Las 4 familias de test que usa el proyecto

Cada familia prueba **una capa** y levanta **una parte** de Spring. Esta tabla la saqué de los imports reales de tus archivos:

Familia	Qué levanta	Qué NO levanta	Para qué	Ejemplos en tu repo
Mockito puro <code>@ExtendWith(MockitoExtension.class)</code>	Nada. Cero Spring.	Spring, la BD, HTTP.	La lógica del servicio con las dependencias inventadas. Rápido, el que más se escribe.	<code>ProductImplTest</code> , <code>PurchaseImplTest</code> , <code>SaleImplLifecycleTest</code> , <code>InputValidatorTest</code>
Datos <code>@DataJpaTest</code>	Solo JPA + los repos. Base de datos en memoria H2 .	Los controllers, la seguridad, los servicios.	Que las consultas JPQL y las relaciones funcionan contra una BD real (H2, no un mock).	<code>ReportQueryTest</code> , <code>UserRepositoryTest</code> , <code>ProductRepositoryTest</code> , <code>ProviderRepositoryTest</code>
Web <code>@WebMvcTest</code>	Solo la capa web: el controller, <code>MockMvc</code> y la seguridad.	Los repositorios y los servicios (van de mentira).	Que el endpoint responde lo correcto y que <code>@PreAuthorize</code> bloquea lo que debe.	Los 12 <code>*ControllerTest</code>
Integración <code>@SpringBootTest</code>	Todo. La aplicación completa.	Nada (por eso necesita una BD de verdad).	Comprobar que las piezas conectadas. Es lento y frágil: por eso solo hay uno.	<code>ComprasApplicationTests</code> (el único que no corre sin BD real)

Por qué esta división es la clave de todo

Un bug puede estar en cualquiera de las capas y **cada familia lo detecta de una forma distinta**. Si solo tuvieras tests de integración, un test lento te diría «algo falló» sin decir dónde. Al combinarlos, cada capa responde su pregunta: «¿la lógica está bien?» (Mockito) «¿la consulta SQL está bien?» (JPA) «¿el JSON y los permisos están bien?» (Web).

5.4 @Mock vs @MockitoBean

Tu proyecto usa los dos y son cosas distintas. Es un cambio de Spring Boot reciente y confunde mucho:

```
// ---- Mockito PURO: la clase se construye a mano. NO hay contenedor de Spring.
@ExtendWith(MockitoExtension.class) // 1. activa Mockito
class ProductImplTest {
    @Mock ProductRepository repo; // 2. "repo" es una Implementation:SIENDO un dato
    @Mock FileStorageService storage;
    @InjectMocks ProductImpl service; // 3. Spring... no, Mockito mete los mocks
    // en el constructor de ProductImpl
}

// ---- WEB: la clase la construye Spring. El bean hay que reemplazarlo.
@WebMvcTest(ProductController.class) // Spring levanta SOLO la capa web
class ProductControllerTest {
    @MockitoBean ProductService service; // 4. le dice a Spring: "en vez del bean
    // real, mete esta Implementation:SIENDO"
    @Autowired MockMvc mvc; // 5. el cliente HTTP de prueba
}
```

	@Mock (+ @InjectMocks)	@MockitoBean
Quién construye la clase bajo prueba	Mockito, a mano	Spring, con el bean sustituido
¿Hace falta arrancar Spring?	No (y por eso son mil veces más rápidos)	Sí, pero solo la rebanada de la anotación (<code>@WebMvcTest</code> o <code>@DataJpaTest</code> , no todo)
¿Qué compruebas?	La lógica: «si el repo devuelve X, el servicio hace Y»	El contrato de HTTP: «con este JSON responde 200 y con este otro 409»

	@Mock (+ @InjectMocks)	@MockitoBean
Nombre del dato falso	La clase que implementa la interfaz, <i>mock</i> en el sentido de «doble de prova»	Un bean de Spring fingiendo ser el real
Si olvidas la anotación	El test falla con <code>NullPointerException</code> : el mocks no se inyectó	Falla al arrancar: <code>NoSuchBeanDefinitionException</code>

En Spring Boot 4, @MockBean quedó obsoleto: por eso en tu proyecto ya no aparece y sí se usa @MockitoBean. Si copiasrís un test de una guía antigua fíjate de la diferencia.

5.5 Las aserciones de AssertJ que más usa el proyecto

`assertThat` es el punto de partida; después encadena métodos. Los que aparecen de verdad en tus tests:

Aserción	Qué comprueba	Ejemplo real del proyecto
<code>assertThat(x).isEqualTo(y)</code>	Valor exacto, con <code>equals</code> .	<code>assertThat(rows.get(0)[1]).isEqualTo(new BigDecimal("10.00"))</code>
<code>assertThat(x).hasSize(n)</code>	Cantidad de elementos de una lista.	<code>assertThat(rows).hasSize(2)</code>
<code>assertThat(x).isNull() / isNotNull()</code>	Que sea (o no) nulo. Ojo: un NULL a propósito es texto , no nulo: hay dos cosas distintas y el proyecto ya tropezó con eso (producto con SKU 'NULL').	<code>assertThat(sinCategoría[1]).isNull()</code> → categoría vacía del producto sin categoría
<code>isEqualByComparingTo(y)</code>	Igualdad de <code>BigDecimal</code> ignorando la escala : 25.00 es igual a 25.	Se usa en los sums de reportes porque la BD devuelve 25 a veces y 25.00 otras.
<code>isTrue() / isFalse()</code>	Un booleano.	Cosas como <code>assertThat(user.isActive())</code>
<code>.as("...")</code>	Mensaje que se lee cuando el assert falla. Muy Ãtil cuando el nÃºmero no se explica solo.	<code>.as("solo cuenta la venta con costo conocido del rango")</code>
<code>assertThatThrownBy(...).isInstanceOf(X.class)</code>	Que la llamada lance una excepci3n concreta.	Todos los 409 de SKU duplicado y de stock insuficiente

5.6 Los 5 verbos de Mockito que te vas a encontrar

```
// 1. given: "si me pides X, te devuelvo Y" (preparar la Implementation:SIENDO)
when(repo.findByBarcode("1111")).thenReturn(Optional.of(gaseosa));
when(repo.save(any())).thenAnswer(inv -> inv.getArgument(0));

// 2. when: la llamada que quieres probar
SaleEntity venta = service.create(request);

// 3. then: verificar que se LLEMO el dato bien
verify(repo).save(argThat(s -> s.getTotal().compareTo(new BigDecimal("10.00")) == 0));

// 4. never: verificar que NO se toco algo <- ESTE es el mas importante en tu repo
verify(productRepository, never()).save(any());

// 5. times(n): cuantas veces se llamo
verify(paymentRepository, times(2)).save(any());
```

Por quÃ© never () es el assert mÃ¡s valioso del proyecto

Un `verify(...).save()` normal demuestra que se guard3 lo que debÃ­a. Un `verify(..., never()).save(any())` demuestra que **no se guard3 nada**, que es justo lo que un `@Transactional` con `rollback` te da gratuitamente. Por eso el `AGENTS.md` lo subraya: no basta con lanzar la excepci3n, hay que probar que no se escribi3 ni una fila. Un test que solo hace `assertThrows` pasarÃ­a aunque el c3digo escribiera la fila justo despuÃ©s de fallar.

5.7 @BeforeEach: el seed

Es el mÃ©todo que corre **antes de cada test**. En `ReportQueryTest` (lÃ­neas 43-88) crea categorÃ­a, 2 productos y 4 ventas. Los tests no lo llaman nunca.

```

@BeforeEach
void seed() {
    CategoryEntity bebidas = new CategoryEntity();
    bebidas.setName("Bebidas");
    categoryRepository.save(bebidas);

    ProductEntity gaseosa = new ProductEntity();
    gaseosa.setName("Gaseosa");
    gaseosa.setPrice(new BigDecimal("5.00"));
    gaseosa.setStock(3);
    ...
    //Venta aprobada en efectivo (dia 10)
    saveSale(LocalDate.of(2026, 9, 10, 10, 0), new BigDecimal("10.00"),
        PaymentMethod.CASH, gaseosa, 2);
    //Venta RECHAZADA en credito (dia 25) -> debe EXCLUIRSE de los agregados
    saveSale(..., new BigDecimal("99.00"), PaymentMethod.CREDIT, cafe, 1);
    //Venta ANULADA (V3, dia 28): 500.00 que NO debe contar
    saveSale(..., new BigDecimal("500.00"), PaymentMethod.CASH, cafe, 1, true);
}

```

bullet **Por qué 4 ventas?** Cada una existe para un caso distinto: aprobada, **rechazada** (no cuenta) y **anulada** (no cuenta). Los agregados filtran por APPROVED y por cancelled = false: son 2 ejes ortogonales, por eso hacen falta las dos condiciones.

bullet **Por qué cada test empieza limpio?** Spring Spring Transaction en @DataJpaTest hace rollback automático al final de cada test. Por eso un test puede agregar ventas y el siguiente no las ve.

bullet **Los asserts cruzan tests a propósito.** Todos los tests de ese archivo esperan «2 ventas y 25.00 en total». Agregar una venta al seed rompe 6 tests a la vez. Es el precio de tener un «mundo común», y es la razón por la que un seed grande necesita comentarios: el tuyo los tiene.

5.8 Cómo ejecutar los tests

En Windows, con el Maven empaquetado (no necesitas instalar nada):

```

# La suite COMPLETA sin base de datos real -> 294/294 en verde
.\mvnw.cmd test -Dtest='!ComprasApplicationTests'

# Una sola clase
.\mvnw.cmd test -Dtest=ReportQueryTest

# Varias clases separadas por coma
.\mvnw.cmd test -Dtest=ReportQueryTest,UserRepositoryTest,InputValidatorTest

# Un solo metodo (el # es el separador de metodo en JUnit 5)
.\mvnw.cmd test -Dtest='ReportQueryTest#sumSalesByDay_agrupaPorDiaSoloAprobadas'

# Dejar el codigo limpio y compilar desde cero
.\mvnw.cmd clean test -Dtest='!ComprasApplicationTests'

```

Por qué el -Dtest='!ComprasApplicationTests'

Porque ese test es el único con @SpringBootTest: levanta la aplicación completa y por eso necesita una base de datos PostgreSQL de verdad, con su usuario y su contraseña. Sin ella, falla. No es un defecto del código: es que los otros 294 tests **no dependen de nada externo** justamente por el diseño de la sección 5.3. Por eso -Dtest con H2 en memoria basta para **todo** lo importante.

Dónde ver el resultado de un fallo:

```
target/surefire-reports/com.erikjarquin.compras.repository.ReportQueryTest.txt
<- El mensaje REAL, sin el ruido de la consola
```

Ademas el propio editor: en VS Code, la flecha verde que aparece junto al metodo en el archivo .java permite correr solo ese test.

5.9 Cómo escribir un test nuevo (paso a paso)

Elige la familia según la capa, y de ahí el esqueleto sale solo. Ejemplo real: probar que anular una venta en efectivo devuelve el stock.

Paso 1. Decide la familia

Si lo que quieres saber es...	Usa	Por qué
«Si el repo devuelve estos datos, mi servicio calcula esto»	Mockito puro	Rápido y sin base de datos
«Si mi consulta JPQL cuenta lo que creo»	@DataJpaTest	Necesitas una base de datos que ejecute la consulta de verdad
«Si el endpoint devuelve el JSON y el código HTTP»	@WebMvcTest	Necesitas que corra la seguridad y los serializadores
«Si todo junto funciona»	@SpringBootTest	Solo para casos críticos: necesita BD real

Paso 2. El esqueleto de un test de servicio (el más común)

```

package com.erikjarquin.compras.service;                // mismo package que la clase

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.*; // o AssertJ
import static org.mockito.Mockito.*;           // atajos: when/verify/never

@ExtendWith(MockitoExtension.class)
class VentaImplTest {

    @Mock    SaleRepository      repo;        // la dependencia que se inventa
    @Mock    ProductRepository   productoRepo;
    @InjectMocks VentaImpl      servicio; // la clase bajo prueba

    @Test
    void anular_devuelveElStock() {
        // GIVEN: una venta de 3 gaseosas, con stock actual 10
        ProductEntity gaseosa = new ProductEntity();
        gaseosa.setStock(10);
        SaleEntity venta = new SaleEntity();
        venta.setPaymentMethod(PaymentMethod.CASH);
        // ... se arma el detalle con quantity 3
        when(repo.findById(1L)).thenReturn(Optional.of(venta));
        when(productoRepo.findById(9L)).thenReturn(Optional.of(gaseosa));

        // WHEN
        servicio.cancel(1L);

        // THEN: el stock vuelve a 13 Y se guarda
        assertThat(gaseosa.getStock()).isEqualTo(13);
        verify(productoRepo).save(gaseosa);
    }

    @Test
    void anular_conTarjeta_NoSePuede() {
        // GIVEN: la misma venta pero pagada con tarjeta
        // ...
        assertThrows(SaleException.class, () -> servicio.cancel(1L));
        // Y lo ideal: que NO se haya tocado nada
        verify(productoRepo, never()).save(any());
    }
}

```

Nota el detalle de la l a `when( . . . )`: Mockito es **estricto**. Si declaras un stub que el test nunca usa, falla con `UnnecessaryStubbingException`. No es un defecto: es una protecci n contra el stub muerto. Tu `AGENTS.md` documenta el caso («las dos pruebas fallaron al escribirlas y la culpa fue del c digo») y la lecci n fue corregir el assert, no el test.

Paso 3. El esqueleto de un test de controller

```
@WebMvcTest(ProductController.class)           // SOLO esa capa
@Import(SecurityConfig.class)                   // para que @PreAuthorize aplique
class ProductControllerTest {

    @MockitoBean ProductService service;        // el bean real, sustituido
    @Autowired MockMvc mvc;                     // el cliente HTTP
    @Autowired ObjectMapper json;               // para armar los cuerpos

    @Test
    void crearProducto_skuDuplicado_devuelve409() throws Exception {
        // GIVEN: el servicio responde "ya existe"
        when(service.save(any()))
            .thenReturn(new ProductException("El SKU ya existe",
                                             HttpStatus.CONFLICT));

        // WHEN + THEN: se simula un POST real
        mvc.perform(post("/api/local/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"name\":\"Coca\",\"sku\":\"CHOC-500\", \"
+ \"barcode\":\"750123\", \"price\":\"10.00\", \"
+ \"stock\":\"5\"}"))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("PRODUCT_ERROR"))
            .andExpect(jsonPath("$.message").exists());
    }

    @Test
    void crearProducto_sinPermiso_devuelve403() throws Exception {
        mvc.perform(post("/api/local/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"name\":\"Coca\"}"))
            .andExpect(status().isForbidden());
        // Y se comprueba que ni se llega al servicio
        verify(service, never()).save(any());
    }
}
```

Pieza	Para qué sirve
<code>mvc.perform(...)</code>	Simula la petición HTTP completa: método, ruta, cabeceras y cuerpo. Lo que llega al controller es lo que se escribió en el JSON.
<code>jsonPath("\$.campo")</code>	Verifica el JSON que sale , sin importar cómo esté armado por dentro. Es lo que ve el frontend.
<code>@WithMockUser(authorities=...)</code>	Finge un usuario ya autenticado con esos permisos. Evita generar JWT reales y probar el 403 de un permiso ausente sin meterte con tokens.
<code>@Import(SecurityConfig.class)</code>	<code>@WebMvcTest</code> no carga la configuración de seguridad por defecto; sin esto los <code>@PreAuthorize</code> no se aplican y los tests de permiso pasarían siempre.
<code>MockMultipartFile</code>	Para probar la subida de imágenes (POST /products es multipart).

5.10 Tests de regresión: la mitad de tu proyecto

Un test de regresión no prueba una función: **prueba que un bug que ya pasó no vuelve a pasar**. Se escribe **después** de romper algo, y su comentario explica el bug. Tres ejemplos reales de tu repo, con la decisión que los motivó:

Test	Bug que blindó	Qué lo hace especial
confirmarCompra_carreraConOtraPeticion_NoSumaStockDosVeces	Un <code>if(entity.isConfirmed())</code> resuelve el doble clic pero no el doble clic simultáneo.	Simula la carrera: la entidad en memoria dice <code>confirmed = false</code> (o sea, un <code>if</code> la habría dejado pasar), y simula que el <code>UPDATE ... WHERE confirmed = false</code> devuelve 0 filas.
cancelarCompra_confirmada_Lanza409YNoEscribe	Cancelar una compra cuyo stock ya se vendió escribía inventario falso con un 200 OK.	El assert clave es <code>verify(productRepository, never()).save(any())</code> : probar que nada se escribió, no solo que saltó la excepción.
nombreSoloEspacios (InputValidatorTest)	El formulario manda " " cuando el usuario deja el campo vacío, y eso pasa un <code>!= null</code> .	El test existe porque el validador compara contra <code>isBlank()</code> y no contra <code>!= null</code> . Si alguien 'simplifica' el validador, este test se pone rojo.

El hábito de un buen test de regresión

Un test se queda si **pero si alguien lo arregla, se pone rojo** para siempre. Un test de regresión es una red: mientras esté tendida no se nota, y el día que la quitan porque «estorbaba» no sabes que ahí estaba. Por eso el `AGENTS.md` documenta el motivo de cada uno: el **por qué** importa más que el **qué**, porque es lo que decide si el test se conserva o se borra.

5.11 Errores típicos

Error	Síntoma	Cómo se detecta
El assert siempre pasa	El test no prueba nada. Un <code>assertThat(x).isNotNull()</code> sobre algo que nunca puede ser nulo.	Rompe el assert a propósito: el test tiene que fallar antes de escribirlo. Si no puedes hacerlo fallar, no está probando.
El test depende del orden	Pasó porque otro test dejó datos. Sale verde al correrlo solo y rojo en la suite.	Correr la clase sola y en la suite: si el resultado cambia, hay estado compartido.
Probar el método, no el efecto	Hacer <code>assertThat(servicio.calcular()).isEqualTo(50)</code> cuando lo que importa es que el stock bajó.	Preguntarse «si este código estuviera mal, ¿algún assert se pondría rojo?».
Prueba que falla al lanzarse	Si el código bajo prueba lanza excepción, el test se cuelga con un error confuso en lugar de una aserción clara.	Un test debe fallar, nunca reventar. Si aparece una excepción inesperada, el assert no se está probando lo que creías.
Probar el framework	<code>assertThat(repo.findAll()).isNotNull()</code> : eso lo prueba Hibernate, no tu código.	Preguntar «¿qué cambiaría si estuviera mal?» Si la respuesta es «nada», sobra.
Mismo stub declarado dos veces	Sobrescribes el <code>when</code> y el primero queda muerto.	Un <code>when</code> por llamada.

5.12 Checklist antes de dar por buena una prueba nueva

- ¿Su nombre se lee como una frase que explica qué comprueba? (no `test1()` ni `testVenta()`)
- Si el código estuviera mal, algún assert se pondría rojo? Si no, el test no sirve.
- La he probado **en rojo** antes de escribir el código? Si nunca falló, no sabes si funciona.
- Es el tipo de test más rápido posible? Si solo necesito la lógica del servicio, no uses `@SpringBootTest`.
- Uso la base de datos real solo cuando necesito SQL de verdad? H2 es más rápida pero habla otro dialecto.
- Si el test documenta un bug, ¿tiene un comentario que explique el bug y no solo el resultado?
- Corre solo? (`mvn test -Dtest=MiTest`) Si depende de otros, es una bomba de relojería.

bullet ¿Sigue en verde después de mvn clean test completo?

6. La línea //REVISAR de ReportQueryTest

6.1 La línea exacta

```
src/test/java/com/erikjarquin/compras/repository/ReportQueryTest.java

123:      detail.setQuantity(quantity);
124:      //REVISAR ESTA LÍNEA: el total de la venta se reparte entre las unidades vendidas
125:      detail.setUnitPrice(total.divide(BigDecimal.valueOf(quantity), 2));
126:      detail.setUnitCost(unitCost);
127:      detail.setSubTotal(total);
```

Está dentro del helper `saveSale(...)`, el que arma la venta y su renglón. Lo que pregunta el comentario es justo lo que voy a responder: **¿está bien deriving el precio unitario así?**

6.2 Qué hace exactamente

`divide` es la división de `BigDecimal`. La firma que se usa aquí tiene dos parámetros:

```
BigDecimal divide(BigDecimal divisor, int scale)

// divisor = BigDecimal.valueOf(quantity) -> la cantidad de unidades
// scale   = 2                           -> dos decimales

// 0 sea: unitPrice = total / quantity, redondeado (o no) a 2 decimales
```

Ejemplos con los datos reales de este archivo:

```
total = 10.00, quantity = 2 -> 10.00 / 2 = 5.00
total = 15.00, quantity = 3 -> 15.00 / 3 = 5.00
total = 5.00, quantity = 2 -> 5.00 / 2 = 2.50
total = 99.00, quantity = 1 -> 99.00 / 1 = 99.00
total = 500.00, quantity = 1 -> 500.00 / 1 = 500.00

Y el producto tiene su propio precio, que NO interviene:
gaseosa.setPrice(5.00) y unitPrice = 10.00 / 2 = 5.00 <- coincide de casualidad
```

6.3 Las dos formas de divide (y por qué importa)

Firma	Qué hace con decimales que no terminan	Cuándoselo usa
<code>divide(BigDecimal divisor, int scale)</code> (la de la línea 125)	Redondea a <i>scale</i> decimales. Es seguro, pero necesita una regla de redondeo implícita.	Divisiones que casi siempre son exactas o con 2 decimales. Mucho código viejo la usa, y por eso a veces alguien añade un tercer parámetro sin saber muy bien por qué.
<code>divide(BigDecimal divisor, int scale, RoundingMode mode)</code>	Igual, pero con la regla de redondeo explícita . Es la recomendada desde Java 5.	Dinero. Aquí importa decir <code>HALF_UP</code> (redondeo comercial) y no el <code>HALF_EVEN</code> que viene por defecto.

Hay además un tercer detalle: `BigDecimal.divide` lanza `ArithmeticException` (no `ArithmeticException` de `int`, sino de `BigDecimal`) si el divisor es cero. Un `quantity = 0` en un test rompe el **setup** completo, antes de cualquier `assert`.

6.4 Por qué hoy no falla: las 5 divisiones del archivo

Se revisó una por una todas las llamadas a `saveSale`. Todas las divisiones son **exactas** (el resultado termina en 1, 2 decimales):

Línea	total	quantity	Resultado	¿Problema?
69	10.00	2	5.00	No
73	15.00	3	5.00	No
77	99.00	1	99.00	No
86	500.00	1	500.00	No
193	5.00	2	2.50	No

Por eso el comentario se puede quitar tranquilo

Con los datos actuales, la línea 125 no tiene **ningún defecto funcional**: no hay truncamientos silenciosos, no hay redondeos raros, no hay excepciones. Funciona.

6.5 Cuándo sí revienta (y por qué es una bomba diferida)

El problema es de robustez, no de hoy. La línea necesita que **toda** división del archivo sea exacta. Y el día en que alguien agrega una venta con cantidad 3 y total 10 sería:

```
// Si alguien escribe esta venta en el seed:
saveSale(LocalDate.of(2026, 9, 12, 10, 0), new BigDecimal("10.00"),
    PaymentMethod.CASH, gaseosa, 3);    // 3 gaseosas

// la linea 125 hace:
10.00.divide(BigDecimal.valueOf(3), 2)
    |
    v
ArithmeticException: Non-terminating decimal expansion; no exact representable
decimal result.  <- 10 / 3 = 3.3333... que NO tiene fin

Y como el throw ocurre en @BeforeEach (el seed), NO es un fallo de este test:
FALLAN LOS 20 TESTS DEL ARCHIVO a la vez, con un error que no menciona
la linea 125 ni habla de una division.
```

El coste real de esa línea

Un helper mal hecho produce un **fallo en cascada que no señala al código culpable**: quien lo vea tendrá que reconstruir por qué fallaron los 20 tests.

6.6 El detalle importante: unitPrice no participa en ningún assert

Esta es la parte que cambia la respuesta a la pregunta del comentario. Revisándolo: las consultas de ReportQueryTest agregan sale.total y SUM(quantity), nunca unit_price. Y hay una prueba en el mismo archivo:

```
// ReportQueryTest.java:344 -- el otro helper del archivo
private SaleEntity saleConCosto(LocalDate date, ProductEntity product, int quantity,
                                BigDecimal unitCost, boolean cancelled){
    SaleEntity sale = new SaleEntity();
    sale.setTotal(new BigDecimal("50.00"));
    ...
    SaleDetailEntity detail = new SaleDetailEntity();
    detail.setSale(sale);
    detail.setProduct(product);
    detail.setQuantity(quantity);
    detail.setUnitCost(unitCost);
    detail.setSubTotal(new BigDecimal("50.00"));
    // <-- NO asigna unitPrice. Ni una linea.
    sale.setDetails(List.of(detail));
    return sale;
}
```

O sea: **hay ventas en el archivo con unitPrice = null y los tests pasan**. Si unit_price fuera NOT NULL en la base, ese helper no persistiría; como @DataJpaTest con create-drop relaja las restricciones, funciona. Confirma que la línea 125 no prueba nada: está ahí para **rellenar una columna**, no para verificar un cálculo.

La lección de fondo

Un //REVISAR en un test rara vez pregunta por el **valor**: pregunta por la **decisión de diseño**. La pregunta aquí no es «¿está bien el redondeo?» (está) sino «¿por qué un helper de test está *calculando* algo en vez de *recibir* un dato?».

6.7 Las tres formas de arreglarla

Las tres son válidas. Presentadas de la más conservativa a la más limpia:

Opción A — cambiar solo la línea (mínimo)

```
import java.math.RoundingMode;
...
125: detail.setUnitPrice(total.divide(BigDecimal.valueOf(quantity), 2,
126:                                RoundingMode.HALF_UP));
```

A favor	En contra
Un cambio de 2 líneas. Ningún test se toca.	Sigue siendo un cálculo dentro del test : si algún día necesitas una venta con total que no sea divisible, tendrás que pelearte con el redondeo.
Se documenta la intención: en dinero, el redondeo es comercial (HALF_UP), no el HALF_EVEN implícito.	

Opción B — que el helper reciba el dato (recomendada)

```
// en vez de derivarlo, que lo digan quien llama:
private void saveSale(LocalDate date, BigDecimal total, PaymentMethod method,
    ProductEntity product, int quantity) {
    saveSale(date, total, method, product, quantity, false, null);
}

private void saveSale(..., BigDecimal unitCost, BigDecimal unitPrice) {
    ...
    // Si no te lo pasan, usa el precio real del producto:
    detail.setUnitPrice(unitPrice != null ? unitPrice : product.getPrice());
    // ^ 5.00 para la gaseosa, 2.50 para el cafe: ya viene en el seed
}
```

A favor	En contra
Cada venta tendría un precio unitario real (el del producto), no uno derivado de un total.	Hay que tocar las 5 llamadas a saveSale si se elige la variante con parámetro.
El test queda predecible : <code>unitPrice * quantity == total</code> solo si el producto cuesta eso, y si no, es visible.	Ninguna real. Es el mismo criterio que ya usa <code>saleConCosto</code> , que recibe <code>unitCost</code> como parámetro.

Opción C — la más simple

```
// El helper NO calcula nada: usa el precio que el producto ya tiene.
123: detail.setQuantity(quantity);
124: //El precio unitario lo pone el producto del seed (5.00 gaseosa,
125: //2.50 cafe). Este helper solo arma la venta; no decide el precio.
126: detail.setUnitPrice(product.getPrice());
```

A favor	En contra
Una sola línea, sin división, sin redondeo, sin excepción.	<code>unitPrice * quantity</code> puede no cuadrar con <code>sale.total</code> (10.00 / 2 sí cuadra; 5.00 / 2 = 2.50 pero el café vale 2.50, aquí sí).
Consistente con el otro helper, que tampoco calcula nada.	

6.8 Lo que yo haría, y por qué

	Opción A	Opción B	Opción C
Líneas a cambiar	2	6-7	3
Riesgo de fallo en cascada si alguien agrega una venta rara	Medio (arreglado, pero sigue habiendo cálculo)	Ninguno	Ninguno
El test dice la verdad sobre los datos	Más o menos	Sí	Sí
Consistente con el resto del archivo	Parcial	Sí (igual que <code>saleConCosto</code>)	Sí
El helper sigue siendo una 'fábrica de ventas' o se convierte en 'decide el precio'	Fábrica	Fábrica	Fábrica

Mi respuesta al //REVISAR

Opción A si quieres quitar el comentario ya y no tocar más: añadir `RoundingMode.HALF_UP` y cambiar el comentario por uno que explique que el reparto es exacto en todos los datos del seed.

Opción B si prefieres el arreglo de fondo: que el helper reciba el precio en vez de deducirlo. Es la que más se parece al resto del archivo y la que evita que la próxima venta con cantidad 3 reviente los 20 tests.

En los dos casos: **no hace falta cambiar ningún assert**, porque `unitPrice` no participa en ninguno (§6.6). Y en los dos casos hay que verificarlo corriendo `-Dtest=ReportQueryTest`: los tests que usan el seed deben seguir dando 25.00 y 2 ventas.

7. Plan de acción

7.1 Lo que no hay que hacer (resumen)

No hagas esto	Por qué
Tocar el bloque app: del YAML	Las 4 propiedades están en uso; la app no arranca. El aviso es cosmético (§1.8).
Añadir la dependencia del procesador esperando que se solucione el problema	Solo documenta payment.terminal.*; las 4 propiedades de app.* se leen con @Value y el procesador no las ve (§1.4).
Borrar bloques vacíos del pom.xml por el aviso	Maven no reporta ninguno (§2.1). Es ruido del editor.
Mover test compilados a src/test/	Ahí solo va código fuente; un .class se regenera y ahí estorba (§4).
git add . ahora mismo	Hay 8 entradas ?? de target_test-classes/ listas para entrar (§3.5).

7.2 Lo que sí conviene hacer, en orden

Prioridad	Acción	Esfuerzo	Riesgo
1	Añadir target_test-classes/ al .gitignore y sacarlo del índice con git rm -r --cached (§3.8 pasos 1-3)	2 minutos	Cero : no toca código. Es lo único urgente.
2	Crear .vscode/settings.json con YAML_UNKNOWN_PROPERTY: IGNORE (§1.5)	1 minuto	Cero: no se versiona.
3	Decidir qué hacer con la línea 125: opción A o B (§6.8), y verificar con -Dtest=ReportQueryTest	10 minutos	Bajo: no hay que tocar asserts.
4	Compilar con mvnw clean test desde la terminal en lugar de desde el editor, para que no vuelva a aparecer la carpeta	0	Cero
5	Limpiar el historial con git filter-repo (saca el .class del commit 252008e; de paso, purga los secretos que AGENTS.md yaihuá)	Media hora	Reescribe hashes: hacer antes de publicar.

7.3 Verificación de que todo sigue bien

```
# 1. La suite completa, sin base de datos real
.\mvnw.cmd clean test -Dtest='!ComprasApplicationTests'
#   esperado: 294/294 en verde, BUILD SUCCESS

# 2. La carpeta huerfana ya no la rastrea git
git check-ignore -v target_test-classes/ReportQueryTest.class
#   esperado: imprime la regla y sale con código 0

git ls-files target_test-classes | Measure-Object
#   esperado: 0

# 3. Los archivos de teste siguen compilando en el sitio correcto
Get-ChildItem target\test-classes -Recurse -Filter *.class | Measure-Object
#   esperado: 28 archivos, y target\test-classes\com\erikjarquin\compras\...

# 4. El ReportQueryTest sigue verde tras el cambio
.\mvnw.cmd test -Dtest=ReportQueryTest
#   esperado: sin fallos; los asserts siguen viendo 2 ventas y 25.00
```

La comprobación de que verdad importa

El aviso amarillo del YAML y el del pom.xml no tienen verificación posible porque **no son fallos**. La única verificación real es la de arriba: **294/294 tests en verde**. Si eso pasa, el proyecto está bien, sea cual sea el color de los triángulos en el editor.

Apéndice A. Glosario rápido de comandos

Comando	Qué hace	¿Cuándo?
<code>.\mvnw.cmd compile</code>	Compila <code>src/main</code> a <code>target/classes</code> . Rápido.	Para comprobar que el código Java está bien.
<code>.\mvnw.cmd test</code>	Compila y ejecuta los tests.	A diario. Añade <code>-Dtest='!ComprasApplicationTests'</code> para no necesitar base de datos.
<code>.\mvnw.cmd clean test</code>	Borra <code>target/</code> y compila desde cero.	Cuando algo raro pasa: siempre compensa antes derable.
<code>.\mvnw.cmd spring-boot:run</code>	Arranca la aplicación en el puerto 8081.	Para probar de verdad, con la base de datos.
<code>git status</code>	Qué archivos cambió hay.	Antes de commitear. ?? = nuevo sin seguimiento, M = modificado.
<code>git check-ignore -v <ruta></code>	Dime qué regla del <code>.gitignore</code> aplica a esa ruta (o si no aplica ninguna).	Para averiguar si un archivo se va a commitear. Código 0 = ignorado, código 1 = se versiona.
<code>git ls-files <ruta></code>	Lista los archivos de esa ruta que git ya rastrea.	Para ver si algo no deseado ya se subió.
<code>git rm -r --cached <ruta></code>	Saca la ruta del índice sin borrarla del disco.	Para dejar de versionar algo sin perderlo.
<code>git log --oneline -10</code>	Los últimos 10 commits, una línea cada uno.	Para ver el estilo de los mensajes antes de escribir el tuyo.
<code>.\mvnw.cmd validate</code>	Solo lee el <code>pom.xml</code> : no compila nada.	Rápido para ver si Maven se queja del <code>pom.xml</code> . Útil: por eso sabemos que el aviso del pom es del editor.

Apéndice B. Archivos y líneas citados

Todo lo afirmado en este documento es verificable. Estas son las referencias exactas, para que puedas abrirlas:

Afirmación	Archivo:línea
Bloque <code>app</code> : con 4 claves	<code>src/main/resources/application.yaml</code> :89-110
Bloque <code>payment.terminal.*</code>	<code>src/main/resources/application.yaml</code> :116-131
<code>app.frontend-url</code> se lee con <code>@Value</code>	<code>service/impl/AuthImpl.java</code> :39
<code>app.seed-bootstraps</code> en los 4 bootstraps	<code>AdminBootstrap</code> :42, <code>PermissionBootstrap</code> :27, <code>RoleBootstrap</code> :28, <code>RolePermissionBootstrap</code> :51
<code>app.upload-dir</code> y <code>app.upload-url</code>	<code>config/WebConfig.java</code> :27, <code>service/FileStorageService.java</code> :46, <code>mapper/ProductMapper.java</code> :22
Única clase <code>@ConfigurationProperties</code> del proyecto	<code>config/TerminalConfig.java</code> :25-26 (<code>@Component</code> + <code>@ConfigurationProperties("payment.terminal")</code> + <code>@Data</code> de Lombok para los setters)
Selección de la terminal por propiedad	<code>service/impl/TerminalSimulatedImpl.java</code> :34 y <code>TerminalPhysicalImpl.java</code> :44 (<code>@ConditionalOnProperty</code>)
La línea <code>//REVISAR</code>	<code>src/test/java/com/erikjarquin/compras/repository/ReportQueryTest.java</code> :124-125

Afirmación	Archivo:línea
El otro helper del archivo, que no asigna unitPrice	ReportQueryTest.java:344-362 (saleConCosto)
El seed con las 4 ventas (aprobada, rechazada, anulada)	ReportQueryTest.java:43-88 (@BeforeEach seed())
Test de ejemplo con Given/When/Then	ReportQueryTest.java:133-146 (sumSalesByDay_agrupaPorDiaSoloAprobadas)
Ejemplo de @WebMvcTest con @MockitoBean y @Import(SecurityConfig.class)	src/test/java/com/erikjarquin/compras/controller/ProductControllerTest.java
Ejemplo de @DataJpaTest con dialecto H2	src/test/java/com/erikjarquin/compras/repository/UserRepositoryTest.java
Regla que ignora target/	.gitignore:2
Negaciones que des-ignoran target dentro de src/	.gitignore:3-4
No hay regla para target_test-classes/	.gitignore completo: git check-ignore -v devuelve código 1
El .class ya versionado	target_test-classes/com/erikjarquin/compras/repository/ReportQueryTest.class, introducido en el commit 252008e
Bloques vacíos del pom.xml	pom.xml:16 (<url/>), 17-19 (licenses), 20-21 (developers), 22-28 (scm)
<optional>true</optional> de Lombok	pom.xml:36
El bloque de Flyway y su <version> con \${flyway.version}	pom.xml:165-175
El .class huérfano (49 archivos, 0.33 MB)	target_test-classes/ en la raíz del repositorio
Número de clases de test en el proyecto	src/test/: 28 archivos .java en 5 paquetes (config, controller, repository, service, util)
El único test que necesita base de datos real	ComprasApplicationTests.java (@SpringBootTest)

Nota final sobre el método

Las cifras de este documento (4 claves, 28 clases de test, 49 .class, 0.33 MB, 1 archivo versionado, líneas 124-125) se obtuvieron ejecutando comandos de solo lectura sobre tu repositorio el día que se generó este PDF. Si después cambias algo (por ejemplo, si aplicas el plan de §3.8), algunas quedarán desactualizadas y habrá que volver a comprobarlas. El contenido del proyecto puede cambiar; el método de verificar con git check-ignore, git ls-files y mvnw validate no cambia.